UNIVERSITY OF APPLIED SCIENCES
ASCHAFFENBURG

# Data Structure and Algorithm

## Project: Disaster Management Planning Tool of Schilda

Authors:
Umaira Zarina Mohamed Nazari
Phuc Ai Tam Vo
Jayvee Rasos

**Document Name:** DSA_Project_Documentation
**File:**https://lv-gitlab.intern.th-ab.de/s231061/algodat

1 INTRODUCTION 4

1.1 Overview of the Disaster Management Planning Tool 4

1.2 Project Scope and Objectives 4

1.3 Functional Requirements of the Tool 4

1.3 Test Inputs for Functions B-B2, F1-F5 5

FUNCTION B1: LOAD AND DISPLAY CITY MAP 7

2.1 Problem Modelling 7

2.2 Graph Algorithm and Data Structures Used 7

2.3 Input Description 8

2.4 Output Description 8

FUNCTION B2: MODIFY CITY MAP 10

3.1 Problem Modelling 10

3.2 Graph Algorithm and Data Structures Used 10

3.3 Input Description 11

3.4 Output Description 11

FUNCTION F1: REBUILD COMMUNICATION INFRASTRUCTURE 13

4.1 Problem Modelling 13

4.2 Graph Algorithm and Data Structures Used 13

4.3 Input Description 14

4.4 Output Description 15

4.5 Complexity Analysis 17

FUNCTION F2: EVACUATION ROUTES 18

5.1 Problem Modelling 18

5.2 Graph Algorithm and Data Structures Used 18

5.3 Input Description 19

**5.4 Output Description** 19

**5.5 Complexity Analysis** 20

**FUNCTION F3: ROUTE PLANNING FOR THE EMERGENCY SERVICES** 21

**6.1 Problem Modelling** 21

**6.2 Graph Algorithm and Data Structures Used** 21

**6.3 Input Description** 22

**6.4 Output Description** 22

**6.5 Complexity Analysis** 23

**FUNCTION F4: SUPPLY OF EMERGENCY SERVICES** 24

**7.1 Problem Modelling** 24

**7.2 Graph Algorithm and Data Structures Used** 24

**7.3 Input Description** 25

**7.4 Output Description** 26

**7.5 Complexity Analysis** 26

**F5: DEPLOYMENT PLANNING FOR EMERGENCY SERVICE** 28

**8.1 Problem Modelling** 28

**8.2 Graph Algorithm and Data Structures Used** 28

**8.3 Input Description** 29

**8.4 Output Description** 29

**8.5 Complexity Analysis** 30

# 1 Introduction

## 1.1 Overview of the Disaster Management Planning Tool

The Disaster Management Planning Tool is designed to assist the city of Schilda in preparing for and responding effectively to emergencies. This tool enables authorities to visualize, modify, and optimize the city's infrastructure, ensuring an efficient disaster response. By representing the city's layout as a graph, the tool allows for the rebuilding of communication networks, planning evacuation routes, allocating emergency services, and establishing supply points all tailored to the city's needs during a disaster.

## 1.2 Project Scope and Objectives

The Disaster Management Planning Tool is designed to address key disaster management tasks, including rebuilding communication infrastructure, planning evacuation routes, optimizing emergency service routes, and deploying supplies. The tool aims to provide an efficient, graph-based system that assists authorities in making data-driven decisions during a disaster, ensuring resources are allocated optimally.

The key objectives of the project are:

- Rebuilding Communication Infrastructure: Restoring connectivity to critical nodes (e.g., rescue stations, hospitals) with minimal repair costs.
- Planning Evacuation Routes: Determining safe and efficient evacuation paths, taking into account road closures and available transportation resources and ensure enough capacities of the infrastructure for the people to evacuate.
- Optimizing Emergency Service Routes: Identifying the fastest and most reliable routes for emergency services, including hybrid routes that combine both land and water travel.
- Supply of emergency Services : Determine the optimal locations for supply points for relief forces, aiming for a minimization of average distance of all relief forces towards supply point.
- Deploying Supplies and Services: Strategically placing supply points to minimize response time and ensure that resources are available where needed.

## 1.3 Functional Requirements of the Tool

The Disaster Management Planning Tool includes several key functionalities to support efficient disaster management in the city of Schilda:

- Load and Display City Map (B1): Visualize the city's infrastructure using an adjacency matrix to help authorities understand the layout.

- Modify City Map (B2): Update the city map to reflect real-time disaster-related changes, such as road closures, infrastructure damage, and the distinction between land and water routes.
- Rebuild Communication Infrastructure (F1): Use graph algorithms to identify and reconnect critical infrastructure nodes (e.g., hospitals, rescue stations) in the most cost-effective way, taking into account damage and repair costs.
- Plan Evacuation Routes (F2): Calculate safe, efficient evacuation paths considering road closures, available transportation, and other constraints (e.g., capacity of infrastructure and the maximum flow of one road to evacuate people).
- Route Planning for Emergency Services (F3): Determine the fastest, most reliable routes for emergency responders, including hybrid land-water travel options when needed.
- Supply of Emergency Services (F4): Identify optimal locations for setting up supply points to minimize response time and ensure rapid delivery of essential resources to emergency teams.
- Deployment Planning for Emergency Services (F5): Optimize the allocation of emergency personnel and resources to disaster sites, ensuring the right skill sets and equipment are sent to the right locations.

## 1.4 Test Inputs for Functions B1-B2, F1-F5

For Function B1, the test input is provided to read and display the graph. This function works by displaying a graph based on a hardcoded adjacency matrix. No external data is loaded, and the graph is visualized using the adjacency matrix provided ***in the code***.

For Function B2, the test input uses a directed unweighted graph. The adjacency matrix represents a city map, with nodes as key locations (e.g., 'A', 'B', etc.) and edges indicating the roads between these locations. No external data is loaded, and the graph is visualized using the adjacency matrix provided ***in the code***.

Graph_directed_unweighted

```
directed_unweighted_matrix = [
    #A, B, C, D, E, F, G, H, I, J
    [0, 1, 0, 0, 1, 0, 1, 1, 0, 1], # A
    [1, 0, 0, 0, 1, 0, 0, 0, 0, 0], # B
    [1, 0, 0, 1, 0, 0, 1, 1, 0, 0], # C
    [1, 0, 1, 0, 1, 0, 0, 0, 0, 0], # D
    [0, 1, 0, 0, 1, 1, 0, 0, 0, 0], # E
    [0, 1, 0, 0, 1, 0, 0, 0, 0, 1], # F
    [1, 1, 1, 0, 0, 0, 0, 1, 1, 0], # G
    [0, 0, 0, 1, 1, 0, 0, 0, 0, 0], # H
    [0, 0, 0, 0, 0, 0, 1, 1, 1, 0], # I
    [1, 0, 0, 0, 0, 0, 0, 0, 1, 0]  # J
]
```

For Functions F1 to F5, we use our own adjacency matrix, which represents an undirected weighted graph. This matrix is loaded from an external source and serves as the test input for these functions, providing the necessary data structure for graph-related operations.

**Undirected_weighted_graph.txt**

```
  A B C D E F G H I
A 0 20 5 7 7 15 25 6 23
B 20 0 34 25 15 10 4 7 16
C 5 34 0 20 27 10 8 12 15
D 7 25 20 0 3 8 9 20 7
E 7 15 27 3 0 27 23 25 0
F 15 10 10 8 27 0 3 7 21
G 25 4 8 9 23 3 0 17 6
H 6 7 12 20 25 7 17 0 30
I 23 16 15 7 5 21 6 30 0
```

# Function B1: Load and Display City Map

## 2.1 Problem Modelling

The primary objective of Function B1 is to load a city map represented by an adjacency matrix and display it in a graphical format. The adjacency matrix models the relationships (edges) between various nodes (locations) in the city. The nodes represent key locations such as places, while the edges represent roads or connections between these locations.

The function:

- Displays a graphical representation of the city map, where the nodes are locations and the edges represent connections between them.
- Outputs on the console, which adjacency matrix is used to the user, helping to visualize the structure of the city's infrastructure and the connectivity between locations.

## 2.2 Graph Algorithm and Data Structures Used

To achieve the desired functionality, the following graph algorithms and data structures are utilized:

1. Graph Representation:
    - The city map is modelled as a graph, where each node represents a city location (e.g., intersections or landmarks), and each edge represents a road or path between two locations.
2. Adjacency Matrix:
    - The city graph is represented as an adjacency matrix, a 2D array where each entry indicates whether a road exists between two nodes.
    - If there is a connection (or road) between two locations, the matrix entry is non-zero (with weights for weighted graphs).
    - The matrix is square with dimensions corresponding to the number of nodes in the graph.
3. Graph Type:
    - The graph can either be directed or undirected, and it can be weighted or unweighted.
        - Unweighted Graph: In this case, matrix entries will either be 0 (no connection) or 1 (direct connection).
        - Weighted Graph: For weighted graphs, matrix entries represent the cost or distance between nodes (non-zero value indicates the weight of the edge, while 0 typically means no connection).
        - Directed Graph: In a directed graph, the matrix is not symmetric, meaning connections from node A to node B do not imply a reverse connection from B to A.
        - Undirected Graph: In an undirected graph, the matrix is symmetric because the connection between two nodes is bidirectional.

4. NetworkX Library:
    - The NetworkX library is used to create and visualize the graph structure.
        - Nodes are added using G.add_node(node).
        - Edges are created from the adjacency matrix using G.add_edge(node_start, node_end, weight=value) (if the graph is weighted).
    - Graph Visualization: The graph is rendered using spring_layout to position nodes and nx.draw_networkx to display the graph, showing nodes, edges, and edge weights (if any).

## 2.3 Input Description

The function takes the following input:

- Adjacency Matrix: A hard coded 2D list (or NumPy array) representing the city's connectivity. This matrix is pre-defined inside the code and is not loaded dynamically from external files.
    - The matrix is square, with dimensions corresponding to the number of nodes (locations) in the city.
    - The matrix can represent either a weighted or unweighted graph, and it can be directed or undirected.
    - Example test inputs for each graph type: Graph_directed_unweighted, Graph_directed_weighted, Graph_undirected_weighted, Graph_undirected_unweighted
- Example of matrices:
    - Unweighted Graph: Matrix entries are 0 (no connection) or 1 (direct connection).
    - Weighted Graph: Matrix entries represent the weight (cost/distance) of the edge between nodes. A non-zero value indicates a connection, while 0 means no connection.
- The graph type can be:
    - Directed: The matrix is not symmetric, as connections may be one-way.
    - Undirected: The matrix is symmetric, indicating bidirectional connections.

## 2.4 Output Description

The function produces the following outputs:

- Graphical Output: A visual graph is displayed using the NetworkX library. This graphical representation shows:
    - Nodes as locations (e.g., intersections or landmarks).
    - Edges as roads or connections between these nodes.

    - Edge Weights (if any), representing the cost or distance between connected locations.
- Adjacency Matrix Output: The adjacency matrix is printed to the console to allow the user to view the exact connectivity structure of the city. The matrix will show 1 for a direct road connection (unweighted) or the weight of the road (weighted graphs), with 0 indicating no road connection.

# Function B2: Modify City Map

## 3.1 Problem Modelling

The primary objective of Function B2 is to allow users to modify an existing city map, which is represented by a directed unweighted adjacency matrix. The city map is visualized as a directed graph where nodes represent key locations in the city, and edges represent roads between these locations. In an unweighted graph, the presence or absence of a road is represented by a 0 or 1 in the adjacency matrix.

Function B2 supports various operations, including:

- Adding important points: Locations that are not necessarily intersections but need to be marked on the map.
- Marking roads as impassable: This could represent closed roads or blocked routes.
- Specifying the type of road: Identifying whether a road is a land route or a waterway.

Once the map is modified, Function B2 visualizes the city map, displaying roads, impassable roads, and important points with their associated distances where applicable. The output graph helps users understand the city structure and plan their routes accordingly.

## 3.2 Graph Algorithm and Data Structures Used

To achieve the desired functionality, the following graph algorithms and data structures are utilized:

- Graph Representation: The city map is represented as a directed unweighted graph, where locations or intersections are the nodes, and roads between them are the directed edges. Each edge may indicate the presence of a road between two nodes.
- Adjacency Matrix: A 2D array representing the adjacency matrix, where each element indicates the presence or absence of a road between nodes. A value of 1 indicates a road exists, while a value of 0 indicates no road exists between the two locations.
- Graph Type: The graph used in Function B2 is directed (the roads have a direction from one node to another) and unweighted (edges simply represent the existence of roads, without associated costs or distances).
- NetworkX Library: This library is used to create, manipulate, and visualize the graph structure. Edges are added using G.add_edge(node_start, node_end) for unweighted graphs, and nodes are added using G.add_node(node_name).
- Matplotlib Library: Used for visualization, allowing the city map to be displayed with a customized layout, colors for different road types, and important points.

## 3.3 Input Description

The function takes the following inputs:

- Adjacency Matrix: A 2D list (or NumPy array) representing the directed unweighted graph. The matrix is square, and its dimensions correspond to the number of nodes (intersections/locations) in the graph.
    - Unweighted Graph: In an unweighted graph, the adjacency matrix entries are 0 or 1, representing the existence or absence of roads.
- Node Labels (Optional): A list of labels corresponding to the nodes in the graph, representing city locations (e.g., ['A', 'B', 'C', 'D']).
- Important Points: A dictionary where the key is the name of the important point, and the value is a tuple containing the location (node) and an optional distance to the nearest intersection.
- Impassable Roads: A list of pairs of nodes representing roads marked as impassable.
- Road Types: A dictionary where the key is a tuple representing the start and end nodes, and the value is the road type ('land' or 'waterway').

## 3.4 Output Description

City Map Visualization:

When the display_city_map() function is called, the city map is visualized as a graph. The graph displays:

- Nodes: Representing city locations, labelled with their node names (e.g., 'A', 'B', 'C', 'D').
- Edges: Representing roads between the locations, with:
    - Gray lines representing normal roads.
    - Blue lines representing waterway roads (if marked).
    - Red lines representing impassable roads.
- Important Points: If any important points (e.g., hospitals, fire stations) have been added, they are marked and labelled at their respective locations. The label displays the point's name and, if provided, the distance to the nearest intersection.

The graphical display will use Matplotlib for visualization. The appearance of the graph will reflect:

- Road colors based on type (gray for normal, blue for waterway, red for impassable).
- Important points marked on the map with names and optional distances.
- A title indicating the graph type (e.g., "City Map of Schilda (Directed Unweighted Graph)").

Textual Outputs:

When calling functions like add_important_point(), mark_road_impassable(), or add_road_type(), the system will print text confirming the operation. For example:

- Adding an important point:
  - “Important point 'Hospital' added at location 'C' with distance 500 to nearest intersection.”
- Marking a road as impassable:
  - “Road from F to E marked as impassable.”
- Specifying a road type:
  - “Road from A to B set as land.”

Adjacency Matrix Output (if print_adjacency_matrix() is called):

- The current adjacency matrix of the city map will be printed in the following format:

Current Adjacency Matrix for Directed_unweighted:

| | A | B | C | D | E | F | G | H | I | J |
|---|---|---|---|---|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 |
| B | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |
| D | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| E | 0 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| F | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 |
| G | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |
| H | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| I | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 |
| J | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |

Graph Type and Map Representation:

- When the map is displayed, the function will print the type of graph being shown, based on the graph_type parameter (e.g., "directed unweighted graph").

# Function F1: Rebuild Communication Infrastructure

## 4.1 Problem Modelling

The task involves developing a communication infrastructure network for the city of Schilda, which needs to be restored after a disaster. The goal is to rebuild a network between critical infrastructure nodes (such as hospitals, rescue stations, and government buildings) to ensure they can communicate effectively while minimizing the total repair cost.

We model the problem as an undirected weighted graph:

- **Nodes** represent infrastructure locations (e.g., hospitals, rescue stations, government buildings).
- **Edges** represent potential communication links between these infrastructure locations, with weights indicating the repair cost to restore each communication link.

Since the communication infrastructure network is undirected, the connections between nodes are bidirectional. This means that if there is a communication link between node A and node B, it can be used in either direction.

The goal is to find the **Minimum Spanning Tree (MST)** of the graph, which ensures that all nodes are connected with the minimal total repair cost while maintaining connectivity between all critical infrastructure locations.

We use **Prim's Algorithm** to solve this problem, as it is efficient for finding the MST in a graph with a relatively dense structure.

## 4.2 Graph Algorithm and Data Structures Used

Graph Algorithm:

- Prim's Algorithm is used for constructing the Minimum Spanning Tree (MST) of the graph. Prim's algorithm is a greedy algorithm that:
    - Starts from an arbitrary node and progressively adds the smallest edge that connects a node in the MST to a node outside of it.
    - The algorithm ensures the minimal cost for the network by always choosing the least expensive edge at each step.

Data Structures:

1. Graph Representation (Undirected Weighted Graph):
    - The graph is represented using NetworkX, a Python library for graph manipulation. Specifically, we use an undirected graph to represent the communication network, where edges have weights representing repair costs.

    - In an undirected graph, if there is an edge between node A and node B with a weight w, it means both A to B and B to A are valid connections with the same cost w.
2. Min-Heap (Priority Queue):
    - A Min-Heap (implemented with Python's heapq module) is used to efficiently select the smallest edge that connects a node in the MST to a node outside of it.
    - Why Min-Heap?
        - The Min-Heap is a key data structure because it supports efficient insertion and retrieval of the minimum edge. It allows us to maintain the minimum edge at the top of the heap, ensuring fast selection of the next edge to add to the MST.
        - In each iteration, Prim's algorithm extracts the minimum-weight edge and adds it to the MST, while the heap ensures that this process runs in optimal time.
3. Visited Set:
    - A visited set is used to track which nodes have already been included in the MST. This ensures that no node is added to the MST more than once.
4. Edge and Node Attributes:
    - Each node in the graph has a description attribute (such as "hospital," "rescue station," or "government building") for better visualization.
    - Each edge in the graph has a weight attribute, which represents the cost to repair the communication link between two nodes.

## 4.3 Input Description

The input to this function is an undirected weighted adjacency matrix (from the file *undirected_weighted_graph.txt*), representing the communication infrastructure network in the city of Schilda. This matrix captures the possible communication links between infrastructure nodes, with each link having a corresponding repair cost.

- Nodes: The nodes in the adjacency matrix correspond to critical infrastructure locations, such as hospitals, rescue stations, and government buildings.
- Edges: The edges between nodes represent communication links that connect these infrastructure locations.
- Weights: The weights of the edges represent the repair costs associated with restoring the communication link between the two nodes. The higher the weight, the more expensive it is to repair the communication link. This also translates to the higher the cost is, the longer the road is.

Since the communication infrastructure is undirected, the adjacency matrix is symmetric. This means that if there is a communication link from node A to node B with a repair cost w, there is also a link from B to A with the same repair cost w.

- Zero weights in the adjacency matrix indicate that no direct communication link exists between the corresponding nodes.

Test input:

```
  A B C D E F G H I
A 0 20 5 7 7 15 25 6 23
B 20 0 34 25 15 10 4 7 16
C 5 34 0 20 27 10 8 12 15
D 7 25 20 0 3 8 9 20 7
E 7 15 27 3 0 27 23 25 0
F 15 10 10 8 27 0 3 7 21
G 25 4 8 9 23 3 0 17 6
H 6 7 12 20 25 7 17 0 30
I 23 16 15 7 5 21 6 30 0
```

In the above adjacency matrix:

- The first row and column represent Node A, the second row and column represent Node B, and so on.
- The matrix entry adjacency_matrix[A][B] = 20 indicates that there is a communication link between Node A and Node B, and the repair cost for this link is 20.
- A value of 0 in the matrix (e.g., adjacency_matrix[A][A] = 0) indicates that there is no self-link and no need to repair such a link.

This matrix serves as the input for the Prim's algorithm, which computes the Minimum Spanning Tree (MST) of the graph to identify the least-cost set of communication links that should be repaired to restore connectivity between all critical infrastructure locations in Schilda.

## 4.4 Output Description

The output consists of the following

1. Total Cost:
    - The total cost to rebuild the infrastructure is calculated as the sum of the repair costs for all edges included in the MST.
    - Example output displayed in the terminal:

    "The minimum cost to rebuild the infrastructure of Schilda city is: 39"

This output means that the total repair cost for restoring the communication infrastructure in Schilda, by connecting all critical infrastructure nodes (such as hospitals, rescue stations, and government buildings), is **39**.

2. Nodes involved in the MST:
    - The nodes involved in the MST represent all the critical infrastructure locations that are connected via the communication links included in the MST.
    - Example output displayed in the terminal:

"Nodes involved in the MST: {'G', 'B', 'C', 'F', 'A', 'I', 'H', 'E', 'D'}"

This output shows the set of nodes (infrastructure locations) involved in the MST, ensuring that these nodes are all connected with the minimum repair cost.

3. MST Edges:
    - The edges of the MST represent the communication links that should be repaired to connect the critical infrastructure locations while minimizing the total repair cost.
    - Example output displayed in the terminal:

```
Minimum Spanning Tree (MST) edges:
Edge: A - C, Repair cost: 5
Edge: A - H, Repair cost: 6
Edge: A - D, Repair cost: 7
Edge: D - E, Repair cost: 3
Edge: E - I, Repair cost: 5
Edge: I - G, Repair cost: 6
Edge: G - F, Repair cost: 3
Edge: G - B, Repair cost: 4
```

Graph Visualization:

- A visual representation of the graph is generated, where:
    - MST edges are highlighted in green to show the chosen edges with the minimum repair cost.
    - Non-MST edges are shown in light gray.
- The labels on the nodes display the description of each infrastructure node (e.g., "Hospital," "Rescue Station").

## 4.5 Complexity Analysis

Time Complexity:

- Prim's Algorithm with a Min-Heap has a time complexity of O(E log V), where:
  - E is the number of edges (communication links) in the graph.
  - V is the number of nodes (infrastructure locations).
- Why O(E log V)?
  - Each edge is pushed to the heap or updated in the priority queue once, which takes O(log V) time. Since there are E edges, the overall time complexity for heap operations is O(E log V).
  - The algorithm processes each node exactly once, which contributes an additional O(V log V) term, but this is dominated by the O(E log V) term when E > V.

# Function F2: Evacuation Routes

## 5.1 Problem Modelling

In an emergency evacuation scenario, it's crucial to calculate whether the existing infrastructure (emergency shelters) can handle the evacuation process or if additional resources are needed. The goal of this function is to model the evacuation process from collection points to emergency shelters while considering the available infrastructure and capacities.

The problem is modelled as a flow network where:

- Nodes represent locations such as evacuation points, shelters, and other key infrastructure.
- Edges represent the routes between these locations with capacities indicating the maximum number of people that can be evacuated per unit time.

We use a maximum flow approach to calculate whether the current infrastructure is sufficient to evacuate all people from collection points to safe emergency shelters.

The core of the problem is modelled as a maximum flow problem using the Edmonds-Karp algorithm, which is an implementation of the Ford-Fulkerson method for calculating maximum flow in a flow network.

## 5.2 Graph Algorithm and Data Structures Used

Graph Algorithm:

- Edmonds-Karp Algorithm:
    - This algorithm is a specific implementation of the Ford-Fulkerson method for calculating the maximum flow in a network. The algorithm works by finding augmenting paths in the residual graph and pushing flow along those paths.
    - Why Edmonds-Karp: This algorithm is chosen because it is easier to implement than other maximum flow algorithms and guarantees an $O(V * E^2)$ time complexity, which is suitable for the scope of the evacuation routes problem where the number of nodes and edges is moderate.

Data Structures:

- Graph Representation:
    - The graph is represented as a directed graph (DiGraph) using NetworkX's DiGraph class. Each edge in the graph has a capacity attribute representing the maximum number of people that can travel along the route.
    - Adjacency Matrix: The graph is created from an adjacency matrix that specifies the connections between nodes (locations) and the respective edge capacities (maximum flow along that route).
- Flow Dictionary:

    - A dictionary (flow_dict) is used to track the flow of people along each edge of the graph. It stores the flow values for each edge, and this is updated as flow is pushed along augmenting paths.
- Residual Graph:
    - A residual graph is maintained, which tracks the available capacity along each edge. The residual capacity is the remaining capacity after flow has been pushed through an edge. This is crucial for the operation of the Edmonds-Karp algorithm, as we repeatedly find augmenting paths in the residual graph.

## 5.3 Input Description

The function requires the following input data:

- Adjacency Matrix:
    - This matrix represents the connections between nodes (e.g., collection points, shelters) with weights (capacities) representing the maximum flow along each route. The graph is constructed using NetworkX's from_pandas_adjacency function from a CSV file.
- Nodes:
    - Locations such as hospitals, rescue stations, government buildings, evacuation points, shelters, etc., are modelled as nodes. Each node can have associated attributes like descriptions (e.g., hospital, government building) and capacities (e.g., the number of people the shelter can hold).
- Routes:
    - The routes between nodes, such as roads, bus routes, and boat routes, are represented as edges with weights. These weights represent the maximum number of people that can be evacuated along these routes.
- Capacity Constraints:
    - The capacities of shelters, buses, or boats are stored as node or edge attributes. For example, shelters have capacities indicating how many people they can accommodate, while routes have capacities showing how many people can travel along them.

## 5.4 Output Description

The output of the function includes the following:

1. Maximum Flow Value:
    - The total number of people successfully evacuated from collection points to shelters. This value is calculated by finding the maximum flow from the source (evacuation point) to the sink (shelters) using the Edmonds-Karp algorithm.
2. Flow Dictionary:
    - A dictionary that contains the flow values for each edge. Each key is a tuple (u, v) representing an edge between two nodes, and the value is the flow along that edge.

3. Graph Visualization:
    - A graphical representation of the evacuation network showing the nodes (locations), edges (routes), and the flow along each edge. The flow on each edge is labelled, and the nodes are labelled with their capacities and descriptions.

## 5.5 Complexity Analysis

The complexity analysis considers the time complexity of the key algorithm used in the function:

Edmonds-Karp Algorithm:

- BFS (Breadth-First Search):
    - The BFS algorithm is used to find augmenting paths in the residual graph. The time complexity of BFS is $O(|V| + |E|)$, where $|V|$ is the number of nodes(locations) and $|E|$ is the number of edges(routes between locations).
- Main Complexity:
    - Each time an augmenting path is found, we increase the flow by at least 1 unit.
    - For each augmenting path found, at least one edge becomes saturated.
    - Each edge can become saturated at most $|V|$ times, which gives us the maximum number of augmenting paths as $O(|V|\cdot|E|)$

- Total Complexity Calculation
    - **Main loop**: The main loop of the algorithm runs $O(|V|\cdot|E|)$ times. This is because, in the worst case, each augmenting path can saturate an edge, and each edge can be saturated at most $|V|$ times (based on the properties of the Edmonds-Karp algorithm).
    - **BFS (Breadth-First Search)**: In each iteration, we perform a BFS to find the augmenting path. The time complexity of BFS is $O(|V| + |E|)$, where $|V|$ is the number of vertices (nodes) and $|E|$ is the number of edges in the graph.

Thus, the total time complexity is the product of the number of augmenting paths and the time complexity of each BFS search:

$$O(|V|\cdot|E|) * O(|V| + |E|) = O(|V|\cdot|E|^2)$$

Where:

- $|V|$ is the number of nodes (locations).
- $|E|$ is the number of edges (routes between locations).

This is the final time complexity of the function, and it guarantees that the evacuation process will be computed efficiently for reasonable input sizes.

# Function F3: Route planning for the emergency services

## 6.1 Problem Modelling

In the event of a disaster, emergency services must reach the scene of the emergency as quickly as possible. The problem can be viewed as a graph traversal problem where we need to calculate the fastest route to the emergency scene (node F – Emergency Service) while considering the condition that some roads may have become impassable due to the disaster. Additionally, a hybrid route involving both car and boat may be necessary when the scene of the emergency can only be reached by boat (e.g., between nodes E - Boat Rescue and I - Staging Area).

In this case, the scenario forces all routes from other nodes to the target node F to go through the inseparable boat route between E and I. The edge EI represents a waterway that cannot be separated and must always be included in the path.

We need to compute both the regular fastest route using only roads and the hybrid route that requires traveling by both car and boat.

## 6.2 Graph Algorithm and Data Structures Used

To model the problem, we utilize the following components:

1. Graph Representation:
    - The problem is represented as an undirected weighted graph where the nodes correspond to infrastructure locations (e.g., hospitals, rescue stations, boat rescue points), and the edges represent the communication routes or roadways between them with associated costs (weights). The graph is represented using an adjacency list with the help of the NetworkX library.
2. Dijkstra's Algorithm:
    - Dijkstra's Algorithm is used to find the shortest path between nodes in the graph. It works by iteratively selecting the node with the smallest tentative distance from the start node and updating the distances of its neighbours.
    - The algorithm is applied twice for each node to calculate the shortest paths first from the node to E and then from I to the target node F.
3. Priority Queue (Min-Heap):
    - A Min-Heap is used internally in Dijkstra’s algorithm to prioritize nodes based on their tentative distance. This ensures that the node with the shortest distance is processed first, optimizing the algorithm's performance.
4. Hybrid Path Calculation:
    - The critical part of the algorithm is ensuring that all paths to F pass through the inseparable waterway between E and I. This is handled by calculating two potential paths for each node:
        1. Path 1: Node → E → I → Target.

2. Path 2: Node → I → E → Target.
   - The shortest of these two paths is selected for each node.

## 6.3 Input Description

The input to this function is an undirected weighted graph, represented as an adjacency matrix. Each node in the matrix corresponds to a critical infrastructure location, and the weights on the edges represent the repair costs for restoring the roads or waterways between them. Specifically:

- Nodes represent critical infrastructure locations such as hospitals, rescue stations, and boat rescue points.
- Edges between the nodes represent communication links (roads or waterways), and the weights represent the time/cost to traverse these links.
- The matrix is symmetric, reflecting the undirected nature of the graph (i.e., if there is a link from A to B, there is also a link from B to A).

Test input:

```
  A B C D E F G H I
A 0 20 5 7 7 15 25 6 23
B 20 0 34 25 15 10 4 7 16
C 5 34 0 20 27 10 8 12 15
D 7 25 20 0 3 8 9 20 7
E 7 15 27 3 0 27 23 25 0
F 15 10 10 8 27 0 3 7 21
G 25 4 8 9 23 3 0 17 6
H 6 7 12 20 25 7 17 0 30
I 23 16 15 7 5 21 6 30 0
```

In the above adjacency matrix:

- A value of 0 indicates no direct communication link between nodes.
- Positive values indicate a road or waterway between nodes with a repair cost (time or money).

## 6.4 Output Description

The output consists of the shortest paths and their respective distances from all nodes to the target node (F – Emergency Service), while considering the required hybrid route via the inseparable waterway (E to I). Specifically:

1. Shortest Paths: The algorithm returns the shortest path from each node to the target node F, ensuring the paths pass through the inseparable E-I waterway.

2. Path Distance: The total distance (repair cost or time) of each path.

Example output displayed in the terminal:

Distances and paths to F via inseparable E-I waterway:
From A to F: Distance = 21, Path = ['A', 'E', 'I', 'G', 'F']
From B to F: Distance = 26, Path = ['B', 'G', 'I', 'E', 'D', 'F']
From C to F: Distance = 26, Path = ['C', 'A', 'E', 'I', 'G', 'F']
From D to F: Distance = 17, Path = ['D', 'E', 'I', 'G', 'F']
From G to F: Distance = 22, Path = ['G', 'I', 'E', 'D', 'F']
From H to F: Distance = 27, Path = ['H', 'A', 'E', 'I', 'G', 'F']

In this example:

- The algorithm computes and outputs the shortest hybrid route that forces all paths to pass through E-I.

## 6.5 Complexity Analysis

The time complexity of the implemented solution is driven by the Dijkstra's Algorithm, which is run twice for each node in the graph:

1. Dijkstra's Algorithm Complexity:
    - Dijkstra's algorithm runs in O(E log V) time using a priority queue (min-heap), where:
        - V is the number of vertices (nodes) in the graph.
        - E is the number of edges in the graph.
2. Total Complexity:
    - For each node, we run Dijkstra's algorithm twice (once to E and once to I).
    - Therefore, the total complexity for all nodes is O(V * (E log V)), where:
        - V is the number of nodes.
        - E is the number of edges.

# Function F4: Supply of emergency services

## 7.1 Problem Modelling

The goal of this function is to determine the optimal locations for two additional supply points in a disaster relief scenario, alongside an existing supply point. The supply points are needed to provide resources to the relief forces spread across various locations. The locations of existing relief stations and the supply points are modelled as nodes in a graph.

The function aims to minimize the total travel distance from all locations to the nearest supply point and distribute relief resources as evenly as possible across the supply points.

The problem is modelled as a graph, where:

- Nodes represent key locations such as hospitals, rescue stations, government buildings, evacuation points, and supply points.
- Edges represent the connections between these nodes, with weights indicating the travel distance or cost to move between nodes.
- Goal: To minimize the total travel distance from all nodes to the nearest supply point while considering practical constraints such as impassable paths and waterways

The problem is solved using a combination of K-Medoid Algorithm to identify the new supply point locations and Dijkstra's Algorithm to calculate the shortest distances between nodes.

## 7.2 Graph Algorithm and Data Structures Used

Graph Algorithm:

1. K-Medoid Algorithm:
    - The K-Medoid algorithm is used to cluster the nodes (locations) into groups, and two additional supply points are selected as the medoids (representative nodes) of these clusters. The objective is to minimize the total distance from all nodes to their nearest medoid.
    - Steps :
        - Initialization: The existing supply point (G) is fixed, and combinations of two additional nodes are tested to find an optimal starting set of medoids.
        - Dijkstra’s Algorithm : Compute the shortest path distances from each medoid to all nodes.
        - Assignment: Assign each node to the nearest medoid based on the shortest path distance in the graph.
        - Cost Calculation: For each combination of medoids, the total cost (sum of distances from all nodes to their closest medoid) is calculated.
        - Optimization: Adjust medoids and repeat the process until optimal placement is found.

Data Structures:

- Graph Representation:
  - The graph is represented using NetworkX's Graph class, which allows us to model the nodes and edges, as well as to store the weights (distances) for each edge.
- Cluster Representation:
  - Medoids: The medoids represent the supply points. These medoids are stored in a list.
  - Clusters: Each node is assigned to a cluster, and the clusters are represented as a dictionary where the keys are medoids and the values are lists of nodes assigned to each medoid.
- Shortest Path Calculation:
  - Dijkstra's Algorithm is utilized to compute the shortest path length from each node to the selected supply point. The graph's edges have weights corresponding to the travel distance, which are considered in the shortest path calculation.

## 7.3 Input Description

The system requires the following input data:

1. Adjacency Matrix:
   - This matrix represents the graph with connections between nodes. Each element in the matrix contains the travel distance (weight) between two locations. The graph is undirected, and the adjacency matrix is used to create the NetworkX graph object.
2. Nodes, Edges:
   - Locations such as hospitals, rescue stations, government buildings, and evacuation points are modelled as nodes. These are represented in the form of a dictionary with node names and descriptions.
   - Obstacles and Waterways : A list of edges representing impassable paths and waterways.
3. Number of Supply Points (k):
   - The number of additional supply points, in this case, k = 2 for two additional supply points. These points would be chosen appropriately based on the existing nodes
4. Graph Representation:
   - The graph is represented using NetworkX, where each edge has a weight indicating the travel distance between the connected nodes.

## 7.4 Output Description

The output consists of two primary components:

1. Selected Supply Points:
    - Final Medoids: The two final supply points are node A – Hospital and node D – Evacuation Point, besides the existing supply point – node G, which minimize the average distance from all relief stations to the nearest supply point. These are the medoids determined by the K-Medoid algorithm.
2. Distances from Supply Points to Nodes:
    - A detailed list of the distances from each selected supply point (medoid) to the nodes (locations) assigned to them. For each node, the shortest distance to its nearest supply point is displayed.
3. Graph Visualization:
    - A visualization of the graph is provided to visually show the nodes (locations) and edges (routes). The supply points (medoids) are highlighted in yellow, and the other nodes are shown in a different colour. The edges between the supply points and their assigned nodes are highlighted in green.

## 7.5 Complexity Analysis

- Function 1: custom_dijkstra

  Calculate the shortest paths with Dijkstra 's Algorithm and the time complexity is O(E log V) where E is edges and V is vertices.

- Function 2: initialize_medoids
    - Time Complexity is $O(n^2 * (V + E \log V))$
    - This function selects k random existing nodes as initial medoids. It nested loops to try all possible combinations of 2 additional points. With n nodes, there are $O(n^2)$ combinations. In each combination, the assigned clusters (assign_clusters) are calculated with the run time $O(kV + kE \log V)$ and then the total cost are computed based on that combination, which takes $O(kE \log V)$.
    - After calculating the total cost, the combination would be checked and chosen if it has lower cost than the former one.
    - Total complexity: $O(n^2 * (kV + kE \log V))$, where k =3
    - The time complexity of this function simplifies: $O(n^2 * (V + E \log V))$
- Function 3: assign_clusters
    - Time Complexity is $O(kE \log V + kV)$, where k =3
    - For each node (total of n nodes), the function computes the shortest path to each of the k medoids using Dijkstra's algorithm. With Dijkstra's algorithm, it is calculated for each medoids (k * E log V) where V is the number of vertices (nodes) and E is the number of edges. Then, for each node, it finds the closest medoid and this process takes O(kV).
    - Therefore, the total time complexity for this function is $O(kE \log V + kV)$, where k =3, where n is the total number of nodes.
- Function 4: calculate_total_cost

    - Time Complexity is O(kE log V + V).
    - This function calls the function 1, which is custom_dijkstra for each medoid, which takes O(kE log V) and calculate the sums of distances for each cluster, which runs O(V) time.
    - Thus, the total time complexity for this function is O(kE log V + V)
- Function 5: k_medoids
    - This function runs iteratively the initialize_medoids once, which takes $O(n^2 * (V + E \log V))$ and make on final assignment of clusters, which runs O(kE log V + kV)

Total Time Complexity:

Total Time Complexity for one iteration:

$O(n^2 * (V + E \log V))$

Where:

- max_iter is the maximum number of iterations (typically 10).
- n is the number of nodes in the graph.
- k is the number of medoid (supply points).
- V is the number of nodes.
- E is the number of edges.

This complexity considers both the K-Medoid process and the shortest path calculations using Dijkstra's algorithm.

# F5: Deployment planning for emergency service

## 8.1 Problem Modelling

The task involves deploying emergency services personnel from staging areas to deployment sites. Each deployment site has specific requirements in terms of the number of units needed, and staging areas have available units to be sent out.

The problem is modelled as a graph where:

- Nodes represent critical locations (e.g., staging areas, hospitals, rescue stations).
- Edges represent deployment routes between these locations, with weights representing the capacity (the number of units or available resources) along each route.

Initially, an undirected graph is used to represent the network of deployment routes between staging areas and deployment sites, as the connections between nodes are bidirectional. Then, we convert this graph into a directed graph to ensure that we can model the direction of deployment from staging areas to deployment sites, as personnel can only be deployed in one direction (from staging area to deployment site).

## 8.2 Graph Algorithm and Data Structures Used

**Graph Algorithm:**

Breadth-First Search (BFS) is used to find optimal deployment paths between staging areas and deployment sites. Key reasons for choosing BFS include:

- It naturally explores the graph level by level, ensuring a fair distribution of resources.
- BFS guarantees shortest paths between nodes, which is ideal for this type of deployment task.

Why not Dijkstra?

- Dijkstra's algorithm is overkill for this task, as the primary focus is on deployment fairness and unit capacity, rather than purely minimizing path length.

**Data Structures:**

1. Graph Representation:
    - The initial undirected graph is represented by an adjacency matrix.
    - The graph is then converted into a directed graph (DiGraph) using NetworkX to model one-way routes for deployment.
2. Deployment Tracking:
    - A dictionary is used to track the number of units deployed along each edge, ensuring no route exceeds its capacity.

## 8.3 Input Description

The system requires the following input data:

1. Adjacency Matrix:
    - Represents the connections between nodes (staging areas and deployment sites), with capacity values indicating the number of units that can be deployed along each route.
2. Nodes:
    - Locations such as hospitals, rescue stations, and staging areas are modeled as nodes.
3. Edges:
    - Routes between nodes are represented as edges with weights that denote the capacity (number of available units).
4. Deployment Needs:
    - A dictionary that specifies how many units are required at each deployment site.
5. Staging Area Capacities:
    - A dictionary that defines how many units each staging area can provide for deployment.

## 8.4 Output Description

**The output consists of two primary components:**

1. Deployment Results:
    - Total Units Deployed: The number of units deployed to each deployment site, ensuring that the deployment needs are met.
    - Deployment Paths: A detailed list of the paths taken for deployment, showing how many units were deployed along each path.
2. Graph Visualization:
    - A visual representation of the deployment network, showing the nodes (locations) and edges (routes). The deployment paths are highlighted in red to show which routes were used for deployment. Each edge shows the flow of deployed units (e.g., 15/20 meaning 15 units deployed out of a 20 unit capacity).

    - Edge Colour Coding :

        1. Black for Impassable edges

        2. Blue for Waterway edges

        3. Red for Regular deployment routes

## 8.5 Complexity Analysis

Time Complexity:

1. BFS Function (bfs_deployment)

- Time Complexity: $O(V+E)$ or $O(n+E)$, where:
  - $V$ is the number of nodes (locations).
  - $E$ is the number of edges (routes between locations).

The BFS function explores each node and each edge at most once. This results in a linear complexity in terms of the number of nodes $V$ and edges $E$.

2. Deploy Units Function (deploy_units)

- Time Complexity:
  $O(D \times S \times ((V+E)+V))$
  where:
  - $D$ is the number of deployment sites.
  - $S$ is the number of staging areas.
  - $V$ is the number of nodes.
  - $E$ is the number of edges.
  - $P$ is the path length (number of edges in the deployment path).

The function iterates through each deployment site ($D$) and for each, it iterates through each staging area ($S$). For each staging area, BFS is called to find a deployment path, which has a time complexity of $O(V+E)$. The function also calculates the available capacity along the path, which is done in $O(P)$ time, where $P$ is the number of edges in the path. The iteration over deployment sites is $O(D)$. The iteration over staging areas is $O(S)$. The BFS search for each path is $O(V+E)$. The calculation of capacity along the path is $O(P)$, which is the number of edges in the path.

Thus, the total complexity for the deploy_units function is $O(D \times S \times ((V+E)+V))$, which simplifies to $O(D \times S \times (2V+E))$.

Total Time Complexity:

- Final Complexity: $O(D \times S \times (V+E))$

  After simplifying, we drop constants and lower-order terms, so the final time complexity becomes $O(D \times S \times (V+E))$, which represents the complexity of processing all deployment sites and staging areas, and performing the BFS search for each deployment path.